

CNG KSP Feature Matrix

Every capability observed across shipping CNG Key Storage Providers, with this project's coverage and — where a gap exists — what closing it would take. Generated 2026-09-28.

102	68	14	20	67%	23
Capabilities tracked	Covered	Partial	Not covered	Fully covered	Actionable gaps

How to read this

Status	Meaning	Ev	Evidence for the market claim
Covered	Implemented and exercised by the test suite.	A	Read from source code or an authoritative API reference that enumerates the provider's own algorithm list.
Partial	Implemented, but reachable only through a non-standard identifier or with a stated caveat.	B	Stated in vendor or Microsoft documentation surfaced through search; primary page not read in full.
Not covered	Absent. The Gap column says what closing it would take.	C	Drawn from HSM datasheets describing the device rather than its KSP. Treat as a lead, not a fact.

Effort on gaps: S = a few lines to a day · M = days · L = a substantial piece of work. **N/A** marks a deliberate position or an external blocker rather than a backlog item — those are excluded from the actionable-gap count.

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
PROVIDER INTERFACE							
IFACE-01	GetKeyStorageInterface export	Sole exported symbol; returns the KSP function table	All providers	A	Covered	—	—
IFACE-02	OpenProvider / FreeProvider	Provider handle lifecycle	All providers	A	Covered	—	—
IFACE-03	GetProviderProperty	Name / Version / ImplType	All providers	A	Covered	—	—
IFACE-04	SetProviderProperty	Provider-level configuration	Vendor-specific	C	Partial	NCRYPT_PIN_PROPERTY is implemented: the PIN is handed to the session layer, and because sessions open lazily a PIN set before the first cryptographic call is the one used to log in. Token selection is read-only through this call by design — the session pool is already bound to a slot by then and PKCS#11 cannot move a session between tokens, so accepting the value would be worse than refusing it. Choose the token with SOFTHSM2_SLOT or SOFTHSM2_TOKEN_LABEL and read it back with KSP_SLOT_PROPERTY.	M
IFACE-05	NotifyChangeKey	Key-change notification handle	Microsoft KSP	A	Partial	Was a no-op returning ERROR_SUCCESS with the caller's HANDLE never written, so an application registering for notification was told it had succeeded and then waited on an uninitialised handle. It now validates the provider and the flags, clears *phEvent and refuses registration with NTE_NOT_SUPPORTED; unregistering succeeds because there is nothing to tear down. Real support needs a key-change channel a PKCS#11 token does not have - there is no C_WaitForSlotEvent equivalent for objects, and polling C_FindObjects on a timer would be a fabrication rather than a notification.	L
IFACE-06	GetOperationProperty	Async / overlapped operation state	Microsoft KSP	B	Not covered	Returns NTE_NOT_SUPPORTED. Only needed for overlapped I/O; PKCS#11 is synchronous so there is nothing to report.	L
IFACE-07	PromptUser	Interactive PIN / consent dialog	Smart card KSPs	B	Not covered	Returns NTE_NOT_SUPPORTED. Would need a UI thread and a window handle from NCRYPT_WINDOW_HANDLE_PROPERTY.	L
IFACE-08	FreeBuffer / FreeObject	Provider-allocated memory release	All providers	A	Covered	—	—

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
IFACE-09	VerifySignature	NCryptVerifySignature through the provider	All providers implement the slot	A	Covered	—	—
TABLE-01	Function table layout matches the SDK	NCRYPT_■KEY_■STORAGE_■FUNCTION_■TABLE slot order and membership	All providers	A	Partial	The table uses designated initialisers, so the SDK header assigns slots by name and a wrong NAME is a build error. Since session 14 eight provider-scoped slots are also signature-checked: their KSP prototype equals the documented public NCrypt one, so test_■function_■table.c assigns each to a pointer type copied from and the build promotes incompatible-pointer-types to an error. That caught KSP_■NotifyChangeKey taking an NCRYPT_KEY_HANDLE where the parameter is HANDLE *pEvent. Key-scoped slots cannot be checked this way - NCryptSignHash takes (hKey, ...) while the slot takes (hProvider, hKey, ...) - so full confirmation still needs ncrypt_provider.h.	S
BUILD-01	Builds for Windows at all	The shipped artefact is a Windows DLL	All providers	A	Partial	Phase 0 fixed all six root causes. MSVC compiles nine of the ten source files clean at /W3 /WX and links test_p11_layer. ksp_main.c cannot be built in hosted CI: ncrypt_provider.h does not exist on a windows-latest runner, established by a dir /s /b across C: and D: that returned only this repo's own test mock, after a WDK install that reported success. Needs a machine that has the header.	M
BUILD-02	Continuous integration	Automated build and test on the target platform	All providers	A	Covered	—	—
ASYMMETRIC RSA							
RSA-01	RSA key generation	2048 / 3072 / 4096 bits	All providers	A	Covered	—	—
RSA-02	RSA 1024 and below	512-1024 bits	MS Software KSP; Utimaco	B	Partial	Build-time opt-in only. KSP_RSA_MIN_BITS defaults to 2048; a legacy build can lower it, but the shipped default still rejects <2048 with NTE_BAD_LEN. Deliberate: silently allowing 1024 would weaken every deployment.	S
RSA-03	RSA above 4096	8192 / 16384 bits	MS Software KSP; Utimaco	B	Covered	—	—
RSA-04	RSA arbitrary size increments	64-bit (MS) or 8-bit (Utimaco) steps	MS Software KSP; Utimaco	B	Covered	—	—
RSA-05	Custom public exponent	Exponent other than 65537	Vendor-specific	C	Covered	—	—
RSA-06	PKCS#1 v1.5 signing	SHA-1 / 256 / 384 / 512	All providers	A	Covered	—	—
RSA-07	PSS signing	SHA-1 / 256 / 384 / 512 with caller salt length	All providers	B	Covered	—	—
RSA-08	PSS with SHA-224	SHA-224 salt and MGF1	Unknown	C	Covered	—	—
RSA-09	PKCS#1 v1.5 decryption	AT_KEYEXCHANGE keys	All providers	A	Covered	—	—
RSA-10	OAEP decryption	SHA-1 / 224 / 256 / 384 / 512	All providers	A	Covered	—	—
RSA-11	OAEP application label	pbLabel / cbLabel passed through	Unknown	C	Covered	—	—
RSA-12	Raw RSA (no padding)	CKM_RSA_X_509	Some HSMS	A	Covered	—	—
ASYMMETRIC ECDSA							
ECDSA-01	ECDSA P-256	secp256r1; 64-byte signature	All providers	A	Covered	—	—
ECDSA-02	ECDSA P-384	secp384r1; 96-byte signature	All providers	A	Covered	—	—
ECDSA-03	ECDSA P-521	secp521r1; 132-byte signature	All providers	A	Covered	—	—
ECDSA-04	Brainpool curves	brainpoolP256r1 / 384r1 / 512r1	Entrust; Utimaco (device)	C	Covered	—	—
ECDSA-05	secp256k1	Bitcoin curve	Entrust (device)	C	Covered	—	—
ECDSA-06	DER to r s conversion	PKCS#11 returns DER; CNG wants raw	All providers	A	Covered	—	—
KEY AGREEMENT							

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
ECDH-01	ECDH P-256 / P-384 / P-521	CKM_ECDH1_DERIVE with CKD_NULL	MS Software KSP; Utimaco	B	Covered	—	—
ECDH-02	SecretAgreement / DeriveKey / FreeSecret	NCRYPT_█SECRET_█HANDL E lifecycle	MS Software KSP; Utimaco	B	Covered	—	—
ECDH-03	BCRYPT_KDF_RAW_SECRET	Raw Z value returned	MS Software KSP	B	Covered	—	—
ECDH-04	BCRYPT_KDF_HASH	SP 800-56A concatenation KDF	MS Software KSP	B	Covered	—	—
ECDH-05	BCRYPT_KDF_HMAC / TLS_PRF / HKDF	Protocol-specific KDFs	MS Software KSP	B	Covered	—	—
ECDH-06	Finite-field Diffie-Hellman	DH 512-4096	MS Software KSP	B	Not covered	Out of scope by choice. ECDH supersedes it and no surveyed HSM KSP exposes it. Would need CKM_█DH_█PKCS_█KEY_█PAIR_█GEN plus CKM_█DH_█PKCS_█DERIVE.	L
ASYMMETRIC EDDSA							
EDDSA-01	Ed25519 signing	64-byte raw signature	None	A	Partial	Works against SoftHSM2 AND verified end to end against Kryptoc (session 10), but uses the non-standard identifier EDDSA_ED25519. CNG defines no EdDSA algorithm ID so no stock application can request it. Nothing to fix in the KSP; blocked on Microsoft.	N/A
EDDSA-02	Ed448 signing	114-byte raw signature	None	A	Partial	Same constraint as EDDSA-01. Note that Ed448 signing did not in fact work until session 10: CKM_EDDSA requires CK_EDDSA_PARAMS for Ed448 and the provider sent none. SoftHSM2 tolerates its absence, so this was invisible until a second token refused it.	N/A
EDDSA-03	X25519 key agreement	Curve25519 ECDH	MS CNG has BCRYPT_EC C_CURVE_25519	A	Covered	—	—
ASYMMETRIC DSA							
DSA-01	DSA signing	512-1024 bits	MS Software KSP	B	Not covered	Deliberately out of scope. Deprecated in modern Windows PKI and absent from every HSM KSP surveyed.	L
POST-QUANTUM							
PQC-01	ML-DSA (Dilithium)	FIPS 204	Microsoft Software KSP (GA late 2025; AD CS ML-DSA enrolment May 2026). Thales Luna firmware 7.9+ and Entrust nShield XC/5/nSaaS ship it in the device; whether their CNG providers expose it is not established	A	Partial	ML-DSA is wired through the KSP and gated on the capability probe: mechanism resolution, CKA_PARAMETER_SET key generation for all three parameter sets, key reopen, and signing. Verified end to end in session 10 against Kryptoc built with its pqc feature over OpenSSL 3.5.8 — generation, reopen by parameter set, and signature at the exact size for ML-DSA-44/65/87. Unreachable on SoftHSM2 2.7.0, which defines the mechanisms and implements none of them. Still Partial for one reason only: public key export is refused, because the CNG post-quantum key blob layout is in a Windows SDK header not available here. That is the same blocker as BUILD-01 and no backend fixes it.	M
PQC-02	ML-KEM (Kyber)	FIPS 203	Microsoft CNG GA (BCrypt side). Thales and Entrust device firmware. No KSP-side exposure established for any provider	B	Not covered	NOT the same blocker as PQC-01. The probe recognises CKM_ML_KEM, so the backend is not what stops this: encapsulation and decapsulation have no slot in the NCRYPT_█KEY_█STORAGE_█FUNCTION_█TABLE this provider implements, so a KSP can hold an ML-KEM key with no entry point to use it. Whether a newer ncrypt_provider.h adds such slots cannot be determined here — same missing header as BUILD-01.	L
PQC-03	LMS / XMSS	Stateful hash-based signatures	Thales Luna, Entrust nShield (device claims only)	C	Not covered	Blocked at the backend, unlike PQC-01: no PKCS#11 v3.2 token surveyed implements LMS or XMSS. Stateful schemes also need one-time-key state persistence the KSP has no model for, and a lost state update forges signatures.	L
SYMMETRIC AES							
AES-01	AES key generation	128 / 192 / 256 bits	None via KSP	A	Covered	—	—
AES-02	AES-ECB	No IV	None via KSP	A	Covered	—	—
AES-03	AES-CBC	16-byte IV	None via KSP	A	Covered	—	—

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
AES-04	AES-CBC with padding	NCRYPT_■PAD_■CIPHER_■F LAG	None via KSP	A	Covered	—	—
AES-05	AES-GCM	12-byte nonce; 128-bit tag; AAD	None via KSP	A	Covered	—	—
AES-06	AES-CTR	16-byte counter block	None via KSP	A	Partial	Works but ChainingModeCTR is a KSP extension. CNG defines no standard counter-mode string so no stock application will select it.	N/A
AES-07	AES-CCM	Authenticated mode	None via KSP	A	Covered	—	—
AES-08	AES-CFB	Cipher feedback	None via KSP	A	Partial	Implemented and verified against a live token for both feedback sizes CNG can reach: the 8-bit default and full-block via MessageBlockLength. Partial because CNG permits any feedback size up to the block, while PKCS#11 defines mechanisms only for 1, 8, 64 and 128 bits; a size with no mechanism is refused rather than silently rounded to a different cipher.	M
AES-09	AES key wrap	RFC 3394 / 5649	Some HSMS (device)	A	Partial	Implemented both ways through BCRYPT_■AES_■WRAP_■KEY_■BLOB, gated on CKM_AES_KEY_WRAP. Unwrap works on any token: a key arrives wrapped under a KEK already on the token and is created sensitive and non-extractable, so wrapping is a way in and not a way back out. Wrap is Partial by consequence, not omission — C_WrapKey answers CKR_■KEY_■UNEXTRACTABLE for every key this provider generates, because they are all CKA_■EXTRACTABLE=FALSE. It is useful only for a key the provider opened rather than created, on a token provisioned with extractable keys.	N/A
AES-10	AES-CMAC	Block-cipher MAC	Some HSMS (device)	A	Covered	—	—
SYMMETRIC MAC							
HMAC-01	HMAC-SHA1 / 256 / 384 / 512	Generic secret keys	None via KSP	A	Partial	Works against SoftHSM2 but HMAC_SHA* are our own identifiers. CNG does HMAC through BCrypt algorithm handles not KSP key algorithms.	N/A
HMAC-02	HMAC-SHA224	SHA-224 variant	None	C	Covered	—	—
SYMMETRIC LEGACY							
3DES-01	3DES / DES	56 / 112 / 168 bits	None via KSP	B	Not covered	Deliberately out of scope. Deprecated and excluded from modern HLK requirements despite SoftHSM2 supporting 14 DES mechanisms.	L
KEY FORMATS							
FMT-01	BCRYPT_RSAPUBLIC_BLOB export	Header + exponent + modulus	All providers	A	Covered	—	—
FMT-02	BCRYPT_ECCPUBLIC_BLOB export	Header + X + Y	All providers	A	Covered	—	—
FMT-03	EdDSA public blob export	Generic ECC magic + raw point	None	A	Covered	—	—
FMT-04	BCRYPT_RSAPUBLIC_BLOB import	Creates a session public key object	Vendor-specific	C	Covered	—	—
FMT-05	BCRYPT_ECCPUBLIC_BLOB import	Creates a session public key object	Vendor-specific	C	Covered	—	—
FMT-06	Private key export	RSAPUBLICPRIVATE / ECCPRIVATE	None (all refuse)	A	Not covered	Refused by design with NTE_NOT_SUPPORTED. Keys are CKA_■EXTRACTABLE=FALSE. This is correct HSM behaviour not a gap.	N/A
FMT-07	Private key import	Wrapping an external private key onto the token	Some HSMS	C	Not covered	Returns NTE_NOT_SUPPORTED. Would need C_CreateObject with private material or C_UnwrapKey. Contradicts the HSM posture; only add if a migration workflow demands it.	L
FMT-08	BCRYPT_KEY_DATA_BLOB	Raw symmetric key material	MS Software KSP	B	Partial	Import is implemented: BCRYPT_■KEY_■DATA_■BLOB creates a CKO_SECRET_KEY via C_CreateObject with CKA_VALUE, AES for 16/24/32-byte material and a generic secret otherwise. Export is refused and will stay refused — every key this provider creates or imports is CKA_■EXTRACTABLE=FALSE, which is the point of the HSM model. Making export work would mean creating extractable keys, weakening every deployment to satisfy one blob type.	N/A

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
FMT-09	PKCS#8 blob	NCRYPT_■PKCS8_■PRIVATE_■KEY_■BLOB	MS Software KSP	B	Not covered	Same constraint as FMT-07: it is a private-key format and keys are non-extractable.	L
FMT-10	OPAQUETRANSPORT blob	Vendor-opaque key transport	Vendor-specific	C	Not covered	Would let keys move between instances of the same provider. Needs a defined wrapping format; no standard to follow.	L
KEY PROPERTIES							
PROP-01	NCRYPT_ALGORITHM_PROPERTY	Read	All providers	A	Covered	—	—
PROP-02	NCRYPT_LENGTH_PROPERTY	Read and write before finalize	All providers	A	Covered	—	—
PROP-03	NCRYPT_KEY_TYPE_PROPERTY	AT_SIGNATURE / AT_KEYEXCHANGE	All providers	A	Covered	—	—
PROP-04	NCRYPT_NAME / UNIQUE_NAME	Key label	All providers	A	Covered	—	—
PROP-05	NCRYPT_EXPORT_POLICY_PROPERTY	Always 0 (non-exportable)	All providers	A	Covered	—	—
PROP-06	NCRYPT_KEY_USAGE_PROPERTY	Signing / decrypt / key agreement flags	All providers	A	Covered	—	—
PROP-07	NCRYPT_ALGORITHM_GROUP_PROPERTY	RSA / ECDSA / ECDH / EDDSA / AES / HMAC	All providers	A	Covered	—	—
PROP-08	NCRYPT_CHAINING_MODE_PROPERTY	Read and write on symmetric keys	MS Software KSP (BCrypt)	A	Covered	—	—
PROP-09	NCRYPT_INITIALIZATION_VECTOR	Read and write on symmetric keys	MS Software KSP (BCrypt)	A	Covered	—	—
PROP-10	NCRYPT_BLOCK_LENGTH_PROPERTY	AES block size	MS Software KSP	B	Covered	—	—
PROP-11	NCRYPT_SECURITY_DESCRIPTOR_PROPERTY	Per-key ACL	MS Software KSP	B	Not covered	Returns NTE_NOT_SUPPORTED. PKCS#11 has no ACL model; would need a side-channel store mapping key labels to SDDL strings.	L
PROP-12	NCRYPT_UI_POLICY_PROPERTY	Strong-key protection prompt	MS Software KSP; smart cards	B	Not covered	Depends on PromptUser (IFACE-07). Same UI-thread requirement.	L
PROP-13	NCRYPT_PIN_PROPERTY	Per-key PIN	Smart card KSPs	A	Covered	—	—
PROP-14	NCRYPT_CERTIFICATE_PROPERTY	Certificate stored alongside the key	Smart card KSPs	A	Covered	—	—
PROP-15	NCRYPT_WINDOW_HANDLE_PROPERTY	Parent HWND for dialogs	Smart card KSPs	B	Not covered	Only meaningful with PromptUser (IFACE-07).	L
PROP-16	NCRYPT_USE_COUNT_PROPERTY	Key usage counter	Some HSMS	C	Not covered	Would need a counter object on the token incremented per operation. SoftHSM2 has no atomic counter primitive.	L
KEY LIFECYCLE							
LIFE-01	CreatePersistedKey	Immediate and deferred generation	All providers	A	Covered	—	—
LIFE-02	FinalizeKey	Deferred generation trigger	All providers	A	Covered	—	—
LIFE-03	OpenKey by name	Asymmetric and symmetric	All providers	A	Covered	—	—
LIFE-04	DeleteKey	Destroys all token objects	All providers	A	Covered	—	—

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
LIFE-05	EnumKeys	Paged enumeration with state	All providers	A	Covered	—	—
LIFE-06	EnumAlgorithms	NCryptEnumAlgorithms support	All providers	B	Covered	—	—
LIFE-07	IsAlgSupported	NCryptIsAlgSupported	All providers	B	Covered	—	—
LIFE-08	Machine vs user key scope	NCRYPT_█MACHINE_█KEY_█FLAG	All providers	B	Partial	NCRYPT_█MACHINE_█KEY_█FLAG is honoured on CreatePersistedKey, OpenKey and EnumKeys: the scope becomes a CKA_LABEL prefix ("m/name" or "u/name"), so a machine key and a user key of the same name are distinct objects instead of silently aliasing. Enumeration filters by scope and reports unprefixed names. This is NAMESPACE, NOT ISOLATION — anyone who can log into the token can read either scope. Real isolation needs an ACL model PKCS#11 does not provide within a slot, and no amount of label work supplies one.	L
OPERATIONAL							
OPS-01	Hardware key protection	Tamper-resistant storage	All commercial providers	A	Not covered	Fundamental: SoftHSM2 is a software token using encrypted SQLite. Only fixable by pointing the KSP at a real HSM which the PKCS#11 abstraction already allows.	N/A
OPS-02	FIPS 140-2/3 validation	Certified cryptographic module	All commercial providers	B	Not covered	Follows from OPS-01. Requires certified hardware and a formal validation programme.	N/A
OPS-03	Authenticode-signed binary	Commercial code-signing certificate	All commercial providers	A	Not covered	Signing is fully tooled: tools/sign_ksp.ps1 signs, timestamps and verifies, taking a certificate from the Windows store, a PFX (password from the environment, never a parameter) or Azure Trusted Signing, and the CI windows job signs automatically once the KSP_█SIGN_█PFX_█BASE64 secret exists. What remains is not engineering: an OV or EV code-signing certificate must be bought by a verified legal entity. Nothing in this repository can supply that, so the row stays Not covered — the binary is unsigned.	N/A
OPS-04	Multiple slots / tokens	Selecting among several tokens	All commercial providers	B	Covered	—	—
OPS-05	Key attestation	TPM / HSM-signed key origin proof	MS Platform Crypto; some HSMs	B	Not covered	No PKCS#11 standard mechanism exists. Vendor-specific and unreachable through a portable PKCS#11 layer.	L
OPS-06	Load balancing / HA	Multiple HSM members	Thales; Entrust	C	Not covered	Handled inside vendor PKCS#11 libraries. Our KSP would inherit it automatically when pointed at such a library; nothing to implement.	N/A
OPS-07	Session pool concurrency	Concurrent NCrypt calls	All providers	A	Covered	—	—
OPS-08	Re-initialisation without restart	Recover from a bad DLL path	Vendor-specific	A	Covered	—	—
OPS-09	PIN caching / re-login	Session recovery after token logout	All commercial providers	C	Covered	—	—
OPS-10	Logging / diagnostics	Trace output for support	All providers	A	Covered	—	—
OPS-13	Runtime capability discovery	Advertised algorithms reflect what the token implements, not a compiled-in list	All commercial providers (implicit — each ships for its own device)	A	Covered	—	—
OPS-14	Backend portability	Any PKCS#11 v2.40+ module can back the provider (KSP_PKCS11_LIB)	Not offered — commercial providers are bound to their own hardware	A	Covered	—	—

Maintaining this document

The source of truth is `softhsm2_ksp/docs/feature-matrix.csv`. One row per capability. To record that a gap has been closed, change that row's *status* to Covered and clear its *gap_solution*; to add a newly observed market feature, append a row with a fresh ID. Then regenerate:

```
python3 generate_feature_matrix.py
```

Every number on page 1 is computed from the CSV at generation time, so the summary cannot fall out of step with the table. Because the CSV is plain text, a change to coverage shows up as a one-line diff in review rather than an opaque binary change.

Actionable gaps by effort: **2 small, 4 medium, 17 large**. Rows marked N/A are excluded — they are deliberate positions (private key export is refused by design) or external blockers (CNG defines no EdDSA identifier; SoftHSM2 is a software token).

A caution on the market columns. An HSM datasheet is not a statement about its CNG provider. AWS CloudHSM performs AES, DES3 and HMAC in hardware, yet its KSP answers NCryptIsAlgSupported for exactly four identifiers. Every grade-C cell here carries that uncertainty. See docs/11-market-comparison.md for the full audit and its sources.